

Twitter Posts – Sentiment Classification Report

1. Project Objective

- Build a sentiment classification model for Twitter data.
- Classify each tweet into one of four classes: Neutral, Irrelevant, Negative, Positive.
- Implement an end-to-end ML pipeline: preprocessing, EDA, feature engineering, model training, and evaluation.

2. Dataset Overview

- Dataset file: twitter_training.csv
- Columns used:
 - TweetedText (input)
 - Sentiment (target)

3. Data Preprocessing

3.1 Missing Values and Duplicates

- Dropped rows with missing TweetedText/Sentiment.
- Removed duplicate tweets.

```
df = df.dropna(subset=['TweetedText', 'Sentiment'])
df = df.drop_duplicates(subset=['TweetedText']).reset_index(drop=True)
```

3.2 Label Encoding

- Neutral → 0
- Irrelevant → 1
- Negative → 2
- Positive → 3

```
label_map = {'Neutral': 0, 'Irrelevant': 1, 'Negative': 2, 'Positive': 3}
df = df[df['Sentiment'].isin(label_map.keys())].copy()
df['label'] = df['Sentiment'].map(label_map).astype(int)
```

3.3 Train/Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    df['TweetedText'],
    df['label'],
    test_size=0.2,
    random_state=42,
```

```
stratify=df['label']
)
```

3.4 Text Cleaning

- Lowercasing
- URL removal
- Mention/hashtag cleanup
- Special character removal
- Stop-word removal
- Lemmatization

```
stop_words = set(stopwords.words('english'))

# Stop word removal may remove words like not/no/never which are crucial for sentiment. So we will retain
them in tweetedtext

for w in ['not', 'no', 'nor', 'n't', 'never']:
    stop_words.discard(w)

lemmatizer = WordNetLemmatizer()

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'http\S+|www\S+', '', text)    # URLs
    text = re.sub(r'@\w+', '', text)            # mentions
    text = re.sub(r'#', '', text)                # hashtag symbol
    text = re.sub(r'^a-z0-9\s', '', text)        # special chars
    text = re.sub(r'\s+', '', text).strip()

    tokens = text.split()
    tokens = [t for t in tokens if t not in stop_words]
    tokens = [lemmatizer.lemmatize(t, pos='v') for t in tokens]
    return ' '.join(tokens)
```

4. Feature Engineering

- Tokenization with Keras Tokenizer
- Vocabulary size: max_words = 10000
- Sequence length: max_len = 200
- Padded sequences for model input

```

max_words = 10000 # We'll consider only the top 10,000 words in the dataset
max_len = 200     # We'll truncate or pad each document to a length of 200 tokens

# Tokenize the text data
tokenizer = Tokenizer(num_words=max_words, lower=True, oov_token="<OOV>")
tokenizer.fit_on_texts(X_train_clean) # fit only on training text

```

5. Exploratory Data Analysis (EDA)

- Mean/median/mode on label distribution
- Sentiment distribution (count and %)
- Top words by sentiment class
- Positive/negative word clouds
- Tweet length vs sentiment boxplots

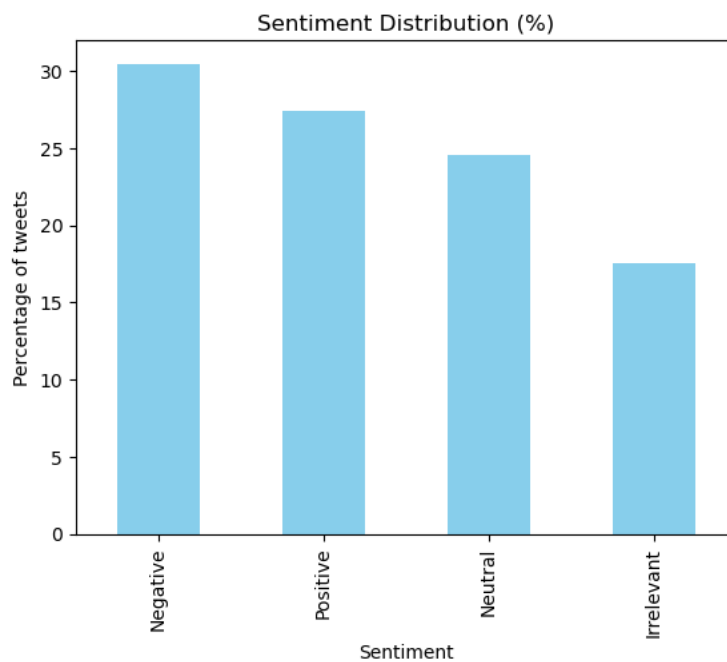
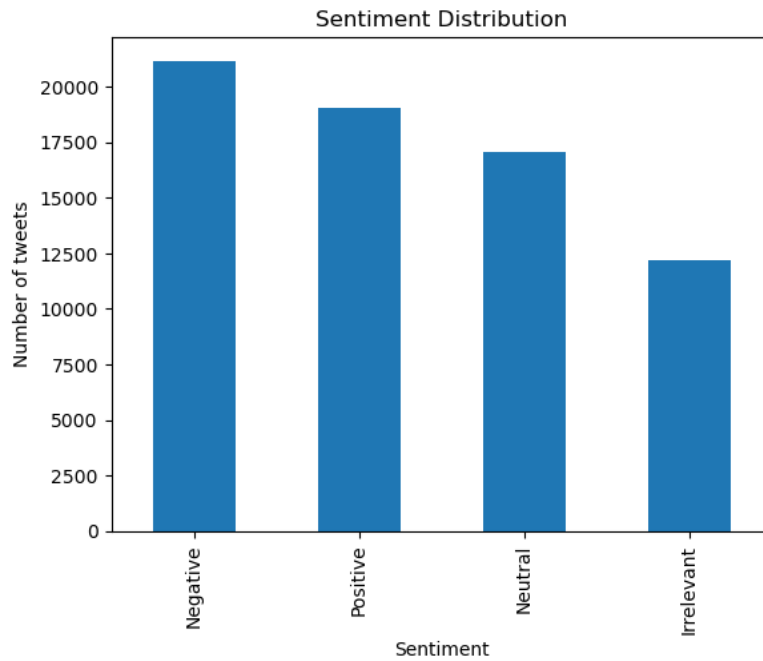
```

print(f"Mean of the Sentiment is which shows the average of the sentiment is : {y_train.mean()}")
print(f"Median of the Sentiment is shows the middle value when data is sorted : {y_train.median()}")
print(f"Mode of the Sentiment provides the sentiment with largest occurrence : {y_train.mode().iloc[0]}")

print(f"Distribution of Sentiment using bar plot")
df['Sentiment'].value_counts().plot(kind='bar')
plt.title('Sentiment Distribution')
plt.xlabel('Sentiment')
plt.ylabel('Number of tweets')
plt.show()

print("Distribution of Sentiment using bar plot (%)")
sentiment_pct = df['Sentiment'].value_counts(normalize=True).mul(100).sort_values(ascending=False)
ax = sentiment_pct.plot(kind='bar', color='skyblue')
plt.title('Sentiment Distribution (%)')
plt.xlabel('Sentiment')
plt.ylabel('Percentage of tweets')
plt.show()

```



```
print(f"The frequency of top words in positive, negative and neutral sentiments" )  
def get_top_words(sentiment, data, top_n=10):  
    subset = data[data['Sentiment'] == sentiment]  
    tokens = []  
    for tweet in subset['TweetedText'].fillna("):  
        if isinstance(tweet, list):  
            tokens.extend([str(w).lower() for w in tweet])
```

```

else:
    tokens.extend(str(tweet).lower().split())
return Counter(tokens).most_common(top_n)

top_positive = get_top_words('Positive', df)
top_negative = get_top_words('Negative', df)
top_neutral = get_top_words('Neutral', df)

print("Top Positive Words:", top_positive)
print("Top Negative Words:", top_negative)
print("Top Neutral Words:", top_neutral)

print(f" Word clouds for positive and negative tweets" )
subset = df.loc[df['Sentiment'] == 'Positive', 'TweetedText']
pos_text = " ".join(subset.fillna("").astype(str))

subset = df.loc[df['Sentiment'] == 'Negative', 'TweetedText']
neg_text = " ".join(subset.fillna("").astype(str))

# Generate Positive Word Cloud
pos_cloud = WordCloud(background_color='white', colormap='Greens').generate(pos_text)
# Generate Negative Word Cloud
neg_cloud = WordCloud(background_color='black', colormap='Reds').generate(neg_text)

plt.title('Positive Word Cloud')
plt.imshow(pos_cloud, interpolation='bilinear')
plt.axis("off")
plt.show()

plt.title('Negative Word Cloud')
plt.imshow(neg_cloud, interpolation='bilinear')
plt.axis("off")
plt.show()

print(f"The relationship between tweet length and sentiment" )
df['tweet_length'] = df['TweetedText'].fillna("").astype(str).str.len()

```

```
ids_to_name = {0: "Neutral", 1: "Irrelevant", 2: "Negative", 3: "Positive"}
df['Sentiment_Label'] = df['Sentiment'].map(ids_to_name)

plt.figure(figsize=(10, 6))

sns.boxplot(x='Sentiment', y='tweet_length', data=df, showfliers=False)

plt.title('Tweet length vs Sentiment (without outliers)')

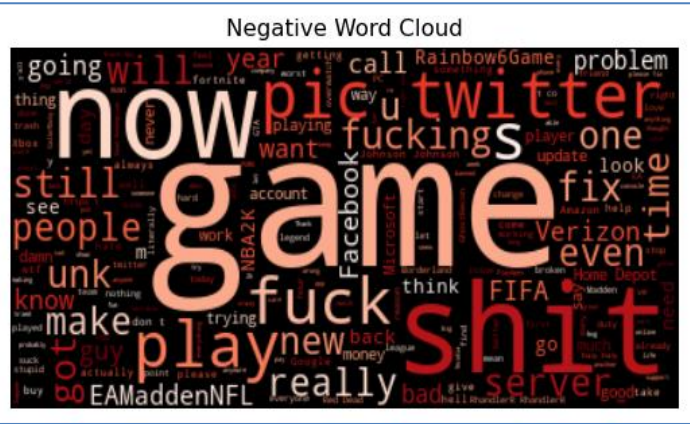
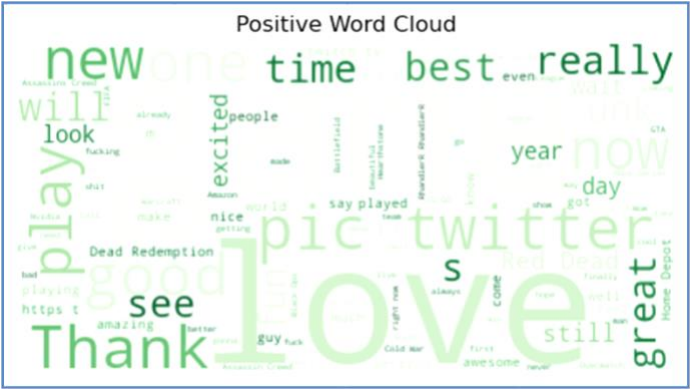
plt.show()

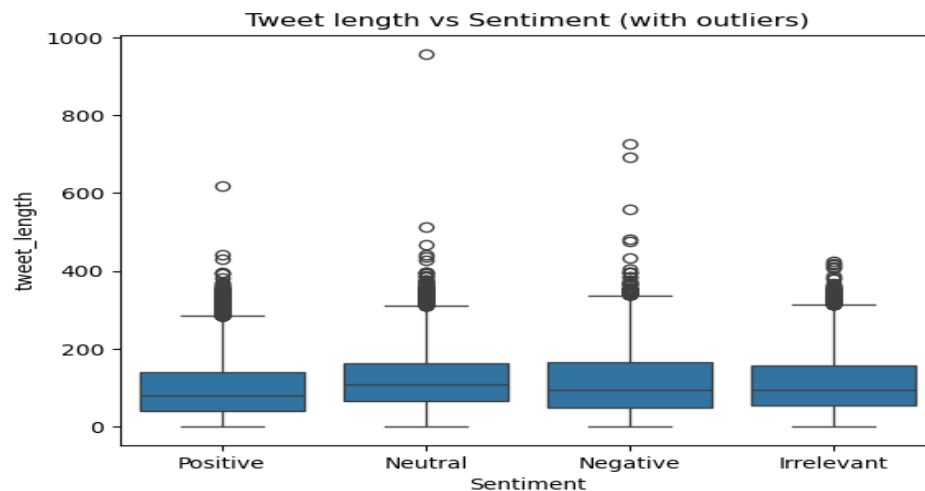
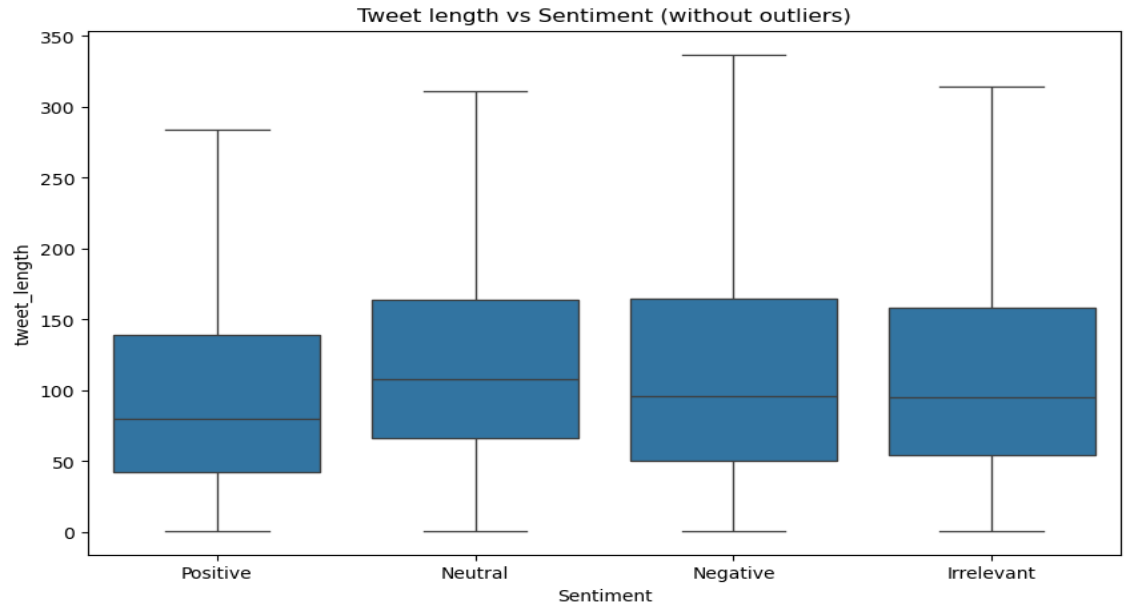
print(f"The relationship between tweet length and sentiment with outliers" )

sns.boxplot(x='Sentiment', y='tweet_length', data=df, showfliers=True)

plt.title('Tweet length vs Sentiment (with outliers)')

plt.show()
```





EDA Highlights

- Negative tweets are generally longer.
- Positive tweets are generally shorter with some outliers.
- Neutral words appear frequently.

6. Model Architecture (BiGRU)

- Input layer (max_len)
- Embedding layer (output_dim=128)
- Bidirectional(GRU(64, return_sequences=True))
- Dropout(0.30)
- Bidirectional(GRU(32))
- Dropout(0.20)
- Dense(64, relu) + Dropout(0.20)

- Output Dense(4, softmax)

```
num_classes = len(np.unique(y_train))
print("\nBuilding the GRU model...")
model = Sequential()
model.add(Input(shape=(max_len,)))
model.add(Embedding(input_dim=max_words, output_dim=128 ))
model.add(Bidirectional(GRU(64, return_sequences=True)))
model.add(Dropout(0.30))
model.add(Bidirectional(GRU(32)))
model.add(Dropout(0.20))
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.20))
model.add(Dense(num_classes, activation='softmax'))
model.compile(optimizer=Adam(learning_rate=0.001), loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
model.summary()
```

7. Training Strategy

- Optimizer: Adam (lr=0.001)
- Loss: sparse categorical crossentropy
- Class weights via compute_class_weight (softened)
- Callbacks:
 - EarlyStopping (patience=3)
 - ReduceLROnPlateau
- Epochs: 15
- Batch size: 128
- Validation split: 0.2

```
# Class weights (handles imbalance)
classes = np.unique(y_train)
weights = compute_class_weight('balanced', classes=classes, y=y_train)
# soften toward 1.0
alpha = 0.5 # 0=no weighting, 1=full balanced
weights = 1 + alpha * (weights - 1)
class_weight = {int(c): float(w) for c, w in zip(classes, weights)}
```



```

callbacks = [
    EarlyStopping(
        monitor='val_loss',
        patience=3,
        restore_best_weights=True,
        verbose=1
    ),
    ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=1,
        min_lr=1e-6,
        verbose=1)
]

```

```

print("\nTraining the GRU model...")
history = model.fit(
    X_train_pad, y_train,
    epochs=15,
    batch_size=128,
    validation_split=0.2,
    class_weight=class_weight,
    callbacks=callbacks,
    verbose=1)

```

8. Model Improvements

- A. With the following baseline model parameters, the accuracy of the model was 30.46% which was very poor.**

The below section highlights the step-by-step changes to improve the performance of the model through various tuning and architecture changes –

Baseline Model -

```

max_words = 10000
max_len = 200
tokenizer = Tokenizer(num_words=max_words, lower=True, oov_token="<OOV>")
model = Sequential()
model.add(Input(shape=(max_len,)))
model.add(Embedding(input_dim=max_words, output_dim=128 )) #,
input_length=max_len))

```

```
model.add(GRU(64, return_sequences=True))
model.add(Dropout(0.5)) # Dropout layer to reduce overfitting
model.add(GRU(32))
model.add(Dense(4, activation='softmax'))
    model.compile(optimizer=Adam(learning_rate=0.001),
        loss='sparse_categorical_crossentropy', metrics=['accuracy'])

print("\nTraining the GRU model...")
history = model.fit(
    X_train_pad, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2,
    verbose=1)
```

1. Following changes were made to the model parameters to improve the performance:
 - a. Drop out was changed from 0.5 to 0.2
 - b. Implemented EarlyStopping and ReduceLROnPlateau
 - c. Changed epochs = 15
 - d. Changed batch size = 128
 - e. After making the above changes, there was no change in the accuracy of the model, and it stayed at **30.46%**
2. As the accuracy is stuck exactly at 30.46% even after architecture/training tweaks, it usually means a data/pipeline bottleneck, not a model-capacity issue. This points to data imbalance.
 - a. We will introduce class weights to handle imbalance. Calculated the weight for each class and assigned higher weight for minor classes and lower weight for major classes.
 - b. After making the above changes, the model accuracy reduced to **27.44%**
3. The drop from 30.46% to 27.44% happened because training focus shifted toward minority classes — expected behavior in imbalanced problems. Further tuning changes performed:
 - a. Removed ReduceLROnPlateau
 - b. increased the patience = 5 for early stopping
 - c. Tried softened weights instead of full 'balanced'
 - d. After making the above changes, the model accuracy came back to **30.46%**
4. Introduced 2 layers of Bidirectional GRU in the model.
 - a. Added a Dense layer with activation function = "relu".
 - b. A bidirectional GRU (Bi-GRU) is better than a simple GRU because it processes data in both forward and backward directions simultaneously. This provides complete context for each time step, significantly improving accuracy in tasks like natural language processing (NLP).
 - c. Brought back ReduceLROnPlateau
 - d. After making the above changes, the model accuracy **improved to 81.98%**

```

num_classes = len(np.unique(y_train))
model = Sequential()
model.add(Input(shape=(max_len,)))
model.add(Embedding(input_dim=max_words, output_dim=128 ))
model.add(GRU(64, return_sequences=True))
model.add(Bidirectional(GRU(64, return_sequences=True)))
model.add(Dropout(0.30))
model.add(Bidirectional(GRU(32)))
model.add(Dropout(0.20))
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.20))
model.add(Dense(num_classes, activation='softmax'))
model.compile(optimizer=Adam(learning_rate=0.001),
loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.summary()

```

B. Transfer Learning to improve Model performance –

Implemented Transfer Learning as requested using GloVe –
glove.twitter.27B.100d.txt.

It was noted that the performance of the model dropped from 81.98% to 71.11%.

The reason could be the data set used for training the model are the tweets pertaining to gaming. Whereas the GloVe tweet dataset is a generic data set.

Since, introducing the transfer learning reduced the performance, I have removed the transfer learning code from the final model design.

The code snippets of the transfer learning code are as follows for reference:

```

# Transfer Learning embeddings from GloVe
EMB_DIM = 100

project_dir = '/Users/ravvarma/Documents/Personal/AI & ML IITM /Week_31/Graded_Project'
glove_txt = os.path.join(project_dir, 'glove.twitter.27B.100d.txt')

if not os.path.exists(glove_txt):
    raise FileNotFoundError(
        f"GloVe file not found at: {glove_txt}\n"
        "Please place glove.twitter.27B.100d.txt in the project folder."
    )

print(f>Loading local GloVe vectors from: {glove_txt}')
embeddings_index = {}
with open(glove_txt, 'r', encoding='utf-8') as f:

```

```

for line in f:
    values = line.rstrip().split(' ')
    word = values[0]

    coefs = np.asarray(values[1:], dtype='float32')
    embeddings_index[word] = coefs

word_index = tokenizer.word_index
vocab_size = min(max_words, len(word_index) + 1)

embedding_matrix = np.random.normal(0, 0.05, (vocab_size, EMB_DIM)).astype('float32')
embedding_matrix[0] = np.zeros((EMB_DIM,), dtype='float32')

hits = 0
for word, i in word_index.items():
    if i >= vocab_size:
        continue

    vec = embeddings_index.get(word)
    if vec is not None:
        embedding_matrix[i] = vec
        hits += 1

```

```

num_classes = len(np.unique(y_train))
print("\nBuilding the GRU model with pretrained embeddings...")
model = Sequential([
    Input(shape=(max_len,)),
    Embedding(
        input_dim=vocab_size,
        output_dim=EMB_DIM,
        weights=[embedding_matrix],
        trainable=False, # Phase 1 frozen
        name='pretrained_embedding'
    ),
    Bidirectional(GRU(64, return_sequences=True)),
    Dropout(0.30),
    Bidirectional(GRU(32)),
    Dropout(0.20),
    Dense(64, activation='relu'),

```

```

Dropout(0.20),
Dense(num_classes, activation='softmax')
])

model.compile(
    optimizer=Adam(learning_rate=1e-3),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']) model.summary()

```

```

print("\nPhase 1: Training with frozen GloVe embedding layer...")
history_phase1 = model.fit(
    X_train_pad, y_train,
    epochs=8,
    batch_size=128,
    validation_split=0.2,
    class_weight=class_weight,
    callbacks=callbacks,
    verbose=1
)

# Phase 2: unfreeze embedding for fine-tuning
print("\nPhase 2: Fine-tuning with unfrozen embedding layer...")
model.get_layer('pretrained_embedding').trainable = True
model.compile(
    optimizer=Adam(learning_rate=1e-5), # very low LR for fine-tuning
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

history_phase2 = model.fit(
    X_train_pad, y_train,
    epochs=5,
    batch_size=128,
    validation_split=0.2,
    class_weight=class_weight,

```

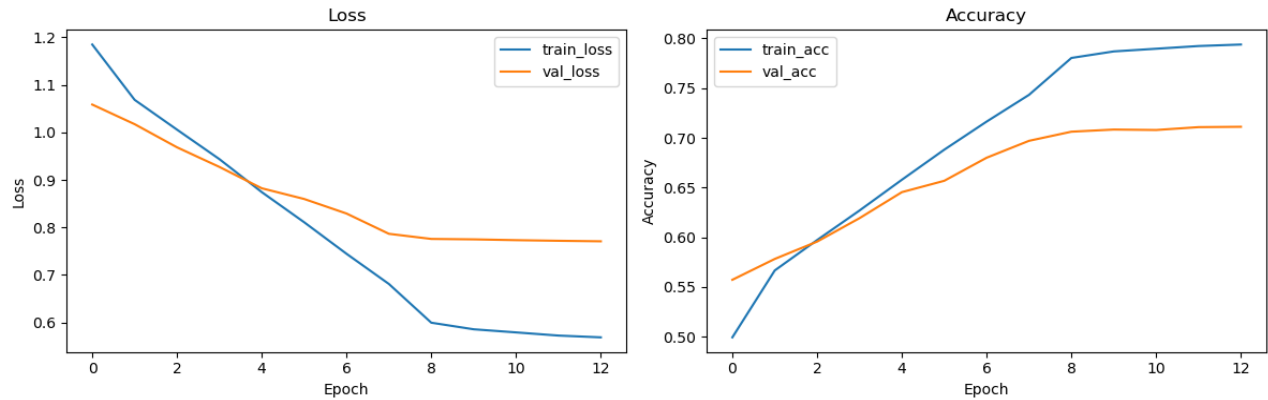
```

callbacks=callbacks,
verbose=1
)
# Merge histories for plotting convenience
history = tf.keras.callbacks.History()
history.history = {}
for k in history_phase1.history.keys():
    history.history[k] = history_phase1.history[k] + history_phase2.history[k]

```

Classification Report:				
	precision	recall	f1-score	support
Neutral	0.74	0.63	0.68	3408
Irrelevant	0.63	0.65	0.64	2443
Negative	0.75	0.79	0.77	4234
Positive	0.70	0.73	0.72	3813
accuracy		0.71		13898
macro avg	0.70	0.70	0.70	13898
weighted avg	0.71	0.71	0.71	13898

Confusion Matrix:
[[2137 308 478 485]
[194 1599 273 377]
[278 291 3353 312]
[295 344 380 2794]]
Precision: 0.7042
Recall: 0.7016
F1-Score: 0.7015



9. Evaluation

Following sections shows the evaluation **details of the model without the transfer learning that provided the best performance:**

- Predictions from `model.predict + argmax`
- Metrics:
 - Accuracy
 - Classification report
 - Confusion matrix

```
print("\nEvaluating the model on test data...")
y_pred_prob = model.predict(X_test_pad)
y_pred_classes = np.argmax(y_pred_prob, axis=1) # Get the predicted class labels
```

```
accuracy = accuracy_score(y_test, y_pred_classes)
print(f'Model Accuracy: {accuracy * 100:.2f}%')

# Display detailed classification metrics for each category
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred_classes,
    labels=[0, 1, 2, 3],
    target_names=['Neutral', 'Irrelevant', 'Negative', 'Positive']
))
```

```
# Generate the confusion matrix
conf_matrix_RF = confusion_matrix(y_test, y_pred_classes)
```



```

print(f'\nConfusion Matrix:\n{conf_matrix_RF}')

precision = precision_score(y_test, y_pred_classes,average='macro')
recall = recall_score(y_test, y_pred_classes,average='macro')
f1 = f1_score(y_test, y_pred_classes,average='macro')

print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")

```

Classification Report:

	precision	recall	f1-score	support
Neutral	0.87	0.76	0.81	3408
Irrelevant	0.79	0.78	0.78	2443
Negative	0.83	0.87	0.85	4234
Positive	0.79	0.84	0.82	3813
accuracy		0.82	13898	
macro avg	0.82	0.81	0.82	13898
weighted avg	0.82	0.82	0.82	13898

Confusion Matrix:

[[2604 168 309 327]

[111 1896 180 256]

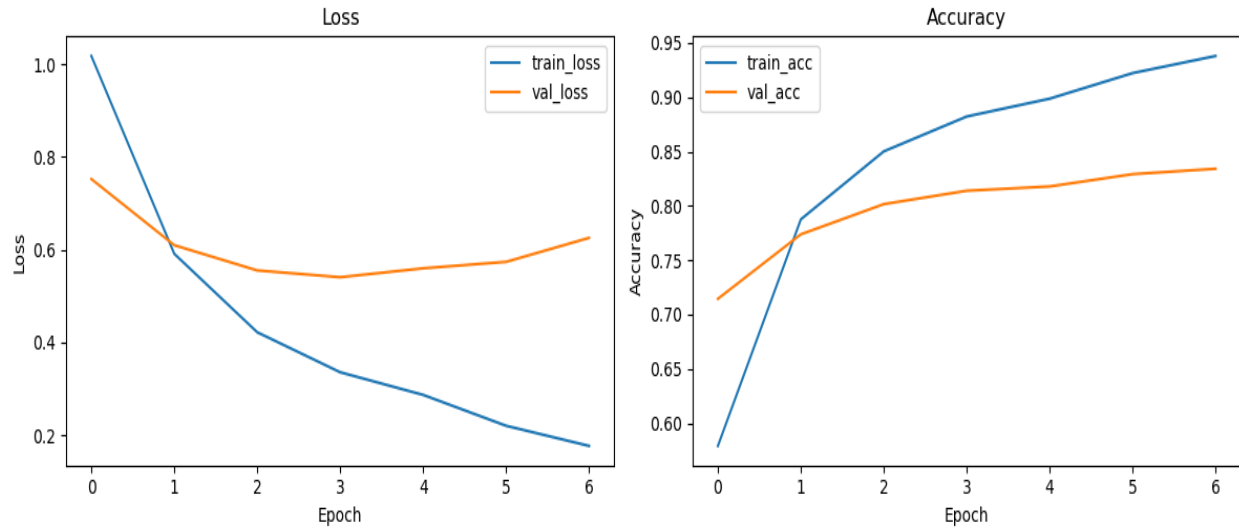
[126 160 3688 260]

[149 172 286 3206]]

Precision: 0.8201

Recall: 0.8130

F1-Score: 0.8153



Reported Performance

- Final test accuracy: ~**81.98%**

9. Visualizations Included

- Sentiment count bar chart
- Sentiment percentage bar chart
- Top-word frequency outputs
- Positive and negative word clouds
- Tweet-length boxplots
- Training/validation loss and accuracy curves

10. Key Takeaways

- Correct preprocessing and label consistency were critical.
- BiGRU improved contextual understanding over basic GRU.
- Class weighting helped class-imbalance handling.
- Pipeline achieved strong performance (~**81.98%**).

11. Future Improvements

- Track macro-F1 as primary metric for imbalance
- Tune `max_len`, `max_words`, and embedding dimension
- Compare with TF-IDF + Logistic Regression baseline
- Explore DistilBERT/BERT for potential further gains

12. Conclusion

An end-to-end sentiment analysis pipeline was built successfully, including preprocessing, EDA, feature engineering, BiGRU modeling, and evaluation. The final model demonstrated strong and stable performance.